Report of mini project step 3 of mandelbrot.

Mandelbrot set problem (same as in first and second miniproject):

The mandelbrot set is a 2 dimensional set in the complex plane. The mandlebrot set is an uncountable set of values of c in the complex plane.
The mandelbrot can be partially calculated with a fixed resolution in a computational way. The algorithm is as follows:

Define fixed region dimensions (x,y)
Generate a grid of complex numbers over the fixed defined region with evenly spread x and y values.
For each point generate value: x+iy as c
For each point now:
    Initialize z = 0
    For n to max iterations (max iteration should be a fixed number)
        Compute: zn+1 = zn 2 + c
        If absolute value of zn+1 > 2: store zn+1 in given point and move to next point

        Else:
        If loop terminates before zn+1 > 2. Store max iterations' value

When loop is finished you have obtained the mandelbrot set in the form of a matrix with iteration counts

The problem is an embarrassingly parallel problem as it is highly paralizable since each value of each grid location can be calculated individually and does not rely on any other data than the complex grid generated equally for all locations.

The problem has many options for implementations. Some will perform better than others in terms of time to execute per loop.

Implementations:

For the third step of the mini project, these given implementations of computing the mandelbrot set has been introduced and performed:

1. Cupy
2. Cuda-Numba

Additionally testing using pytest has been implemented in a test.py script using asserts to unit test function outputs.

The exact implementation is included in the code in the form of a python function as part of a mandelbrot class for storing testing parameters. Performance benchmarks has been documented and is attached in a csv folder. stat_functions.py reads from the csv files and plots the max, mean, min result for each run as well as a scaling analysis plot.

1. Cupy:

Cupy is implemented exactly as the numpy vectorized algorithm using cupy as cp instead of numpy as np in the syntax.

Using this instead allows the same NumPy-style vectorized code to run on the GPU. The initial data transfer from CPU to GPU can add overhead in very large problem sizes as we will see later in the benchmarking results.

2. Cuda-Numba

CUDA Numba is implemented using a custom GPU kernel written with Numba's cuda.jit.

In this function we manually have to declare data transfer from host to device (CPU to GPU) and after that the function can execute the operations on GPU.

Threads per block is defined as a fixed 16x16 meaning each block computes 256 pixels rounding up if the size is not divisible by 16.

blocks_per_grid_x and blocks_per_grid_y calculate how many blocks are needed to cover the full image in each dimension, rounding up if the size is not divisible by 16.

blocks_per_grid combines both dimensions to define the full grid of blocks that will be launched on the GPU.

Inside the kernel function, i, j = cuda.grid(2) computes the global 2D coordinates of the current CUDA thread within the entire grid.

Benchmarking methodology:

The benchmarking utilized timeit.repeat function. This repeat generates a list of runs which the average can be derived from.

To measure the performance when scaling the problem size. The given resolutions of the grid have been tested with some of the implemented functions.

1024x1024
2048x2048
4096x4096
8192x8192

And for some cases:
16384x16384

With max iterations of:
100
200
300
400

And for some cases:
500
600

Each test ran on a local desktop machine.

Due to some functions performing way better than others, some of these functions were tested against even greater problem sizes to properly measure their scaling complexity when the problems become very large.

PC specs:

=== Static System Information ===

System: Windows 10
Machine: x86_64
Processor: x86_64
CPU Brand: AMD Ryzen 9 5950x 16 core processor
CPU Arch: X86_64
CPU Bits: 64
Number of Physical Cores: 16
Number of Logical Cores: 32

=== CPU Cache Sizes ===
L1 Data Cache: 16x32 KBytes
L1 Instruction Cache: 16x32 KBytes
L2 Cache: 16x512 KBytes
L3 Cache: 2x32 MBytes

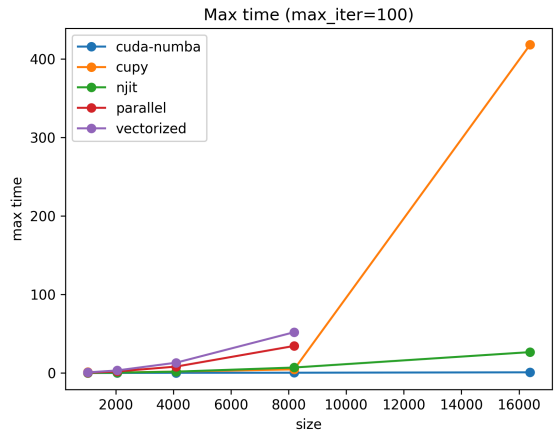
=== Memory ===
Total RAM: 32 GB

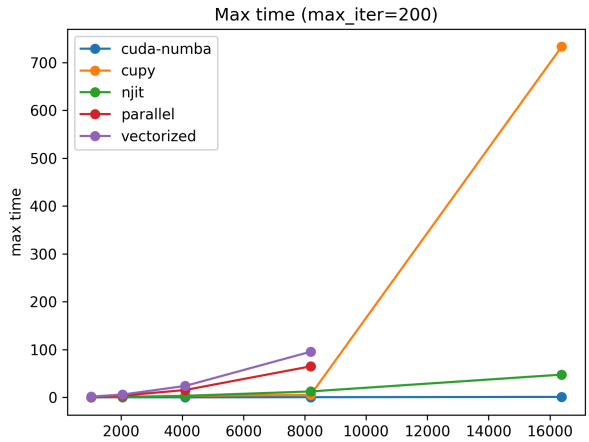
=== GPU System Information ===

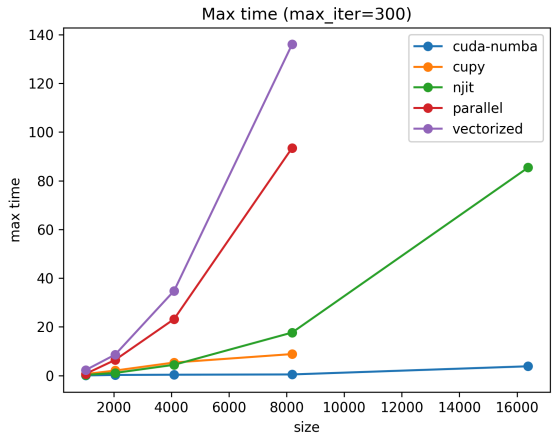
GPU: NVIDIA GeForce RTX 3090
Architecture: Ampere (GA102)
CUDA Cores: 10,496
Base Clock: ~1.40 GHz
Boost Clock: ~1.70 GHz
VRAM: 24 GB GDDR6X
Memory Bandwidth: 936 GB/s
Bus Width: 384-bit
CUDA Compute Capability: 8.6

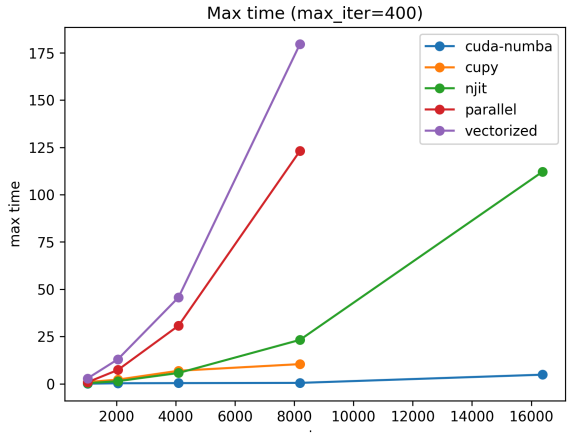
Benchmarking results

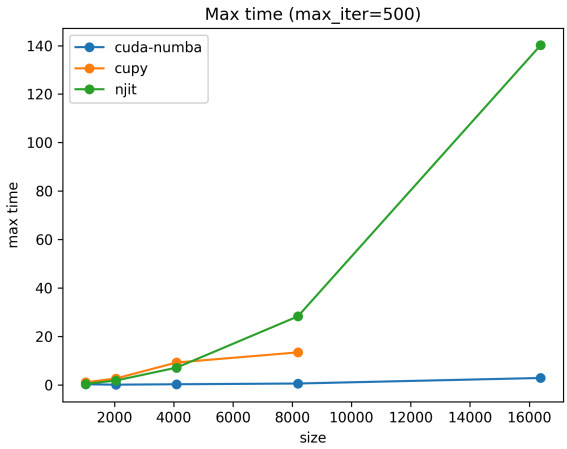
Max time result across all tests 100-600 max iterations:

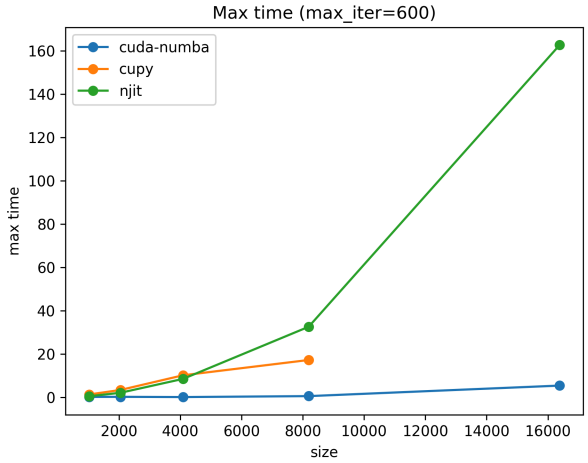




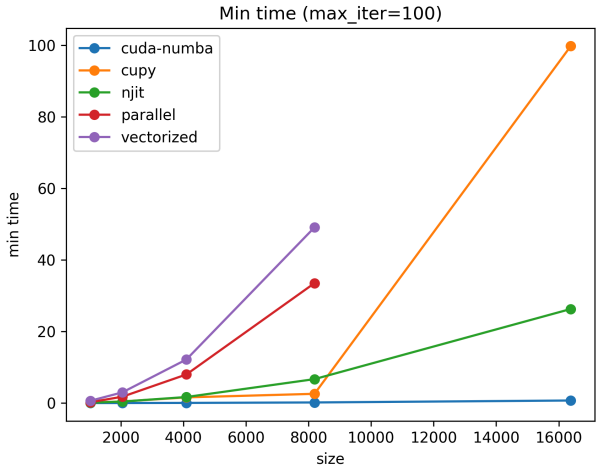


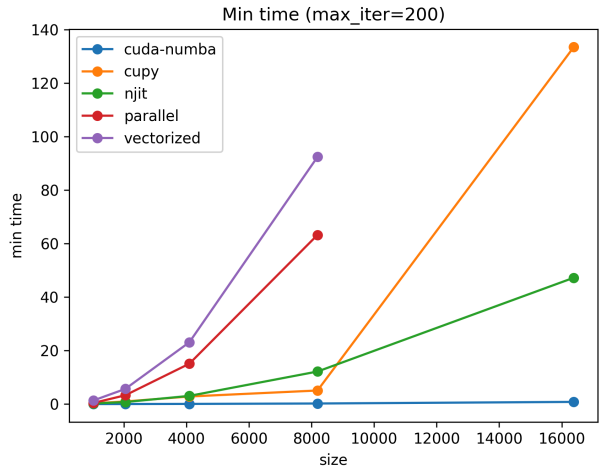


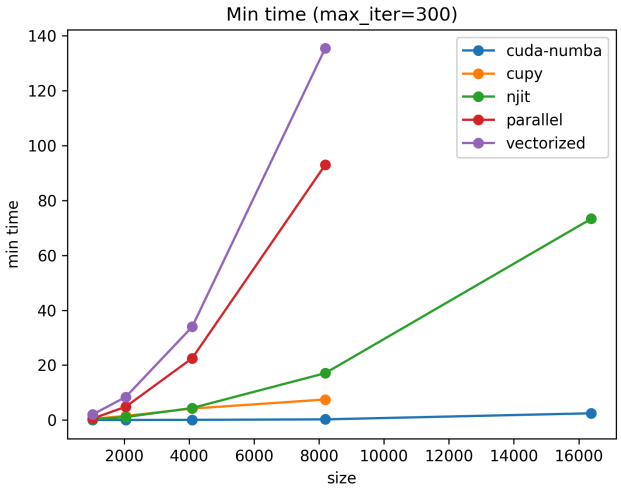


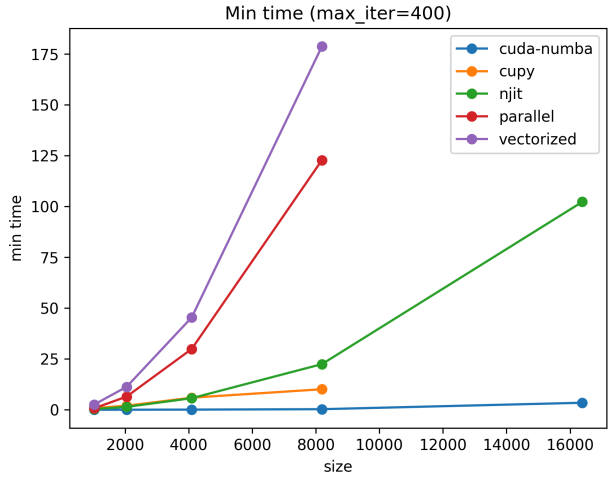


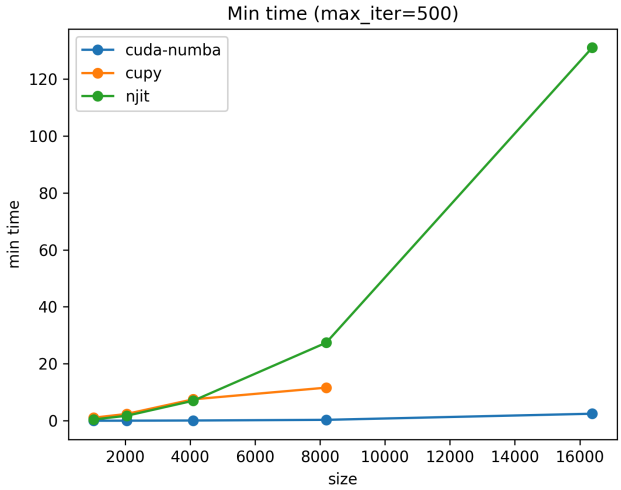
Min result across all tests 100-600 max iterations:

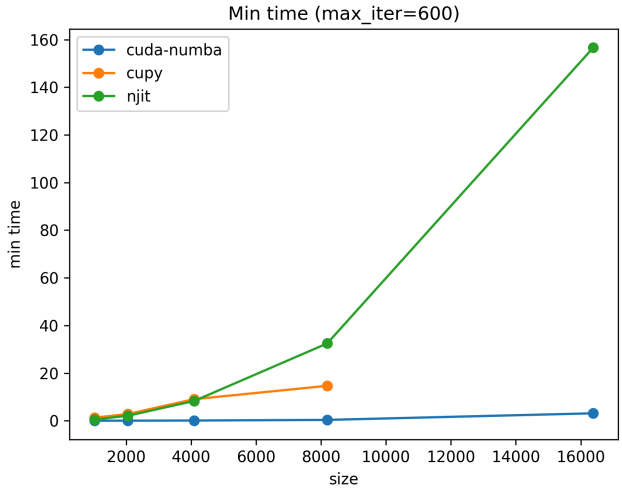




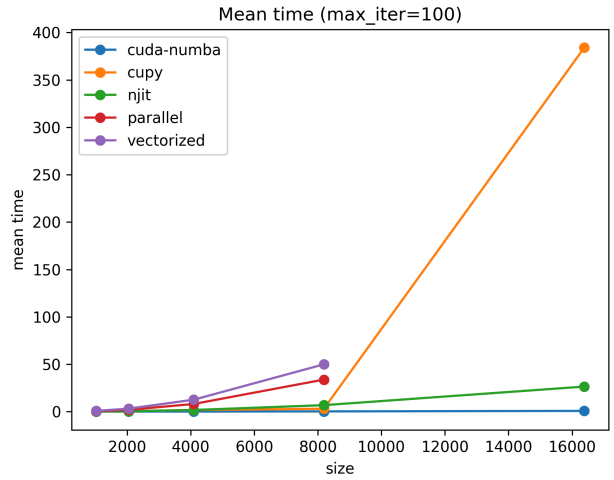


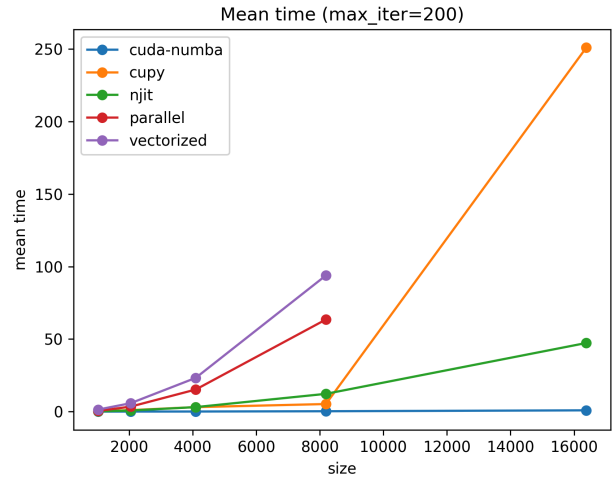


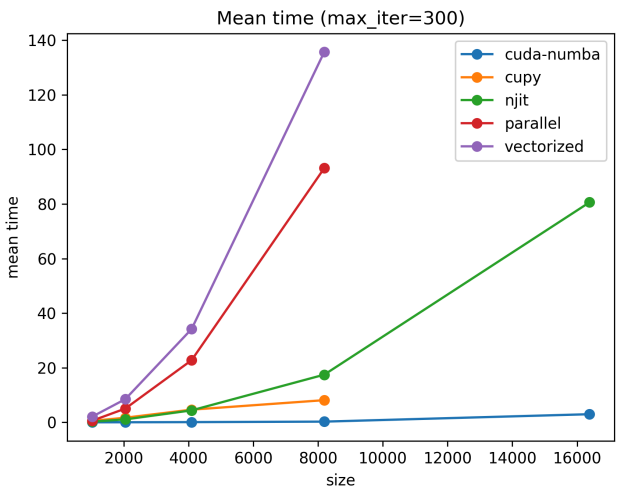


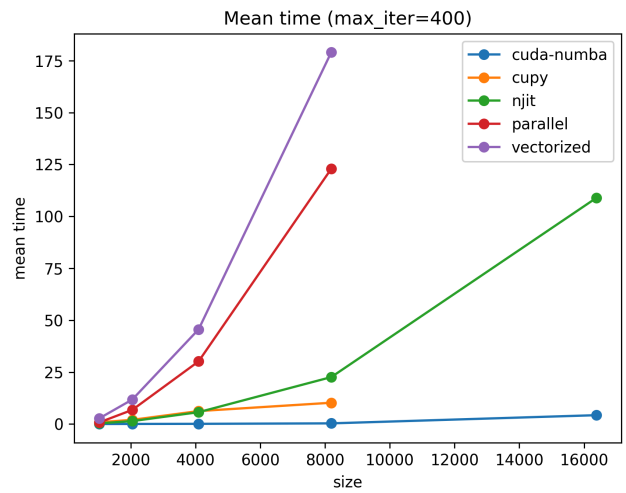


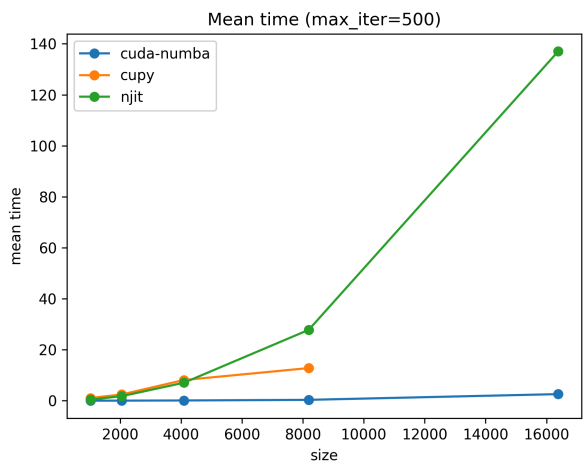
Mean result across all tests 100-600 max iterations:

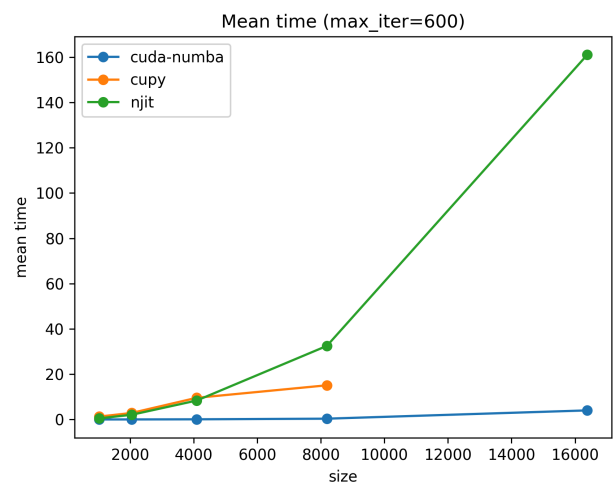




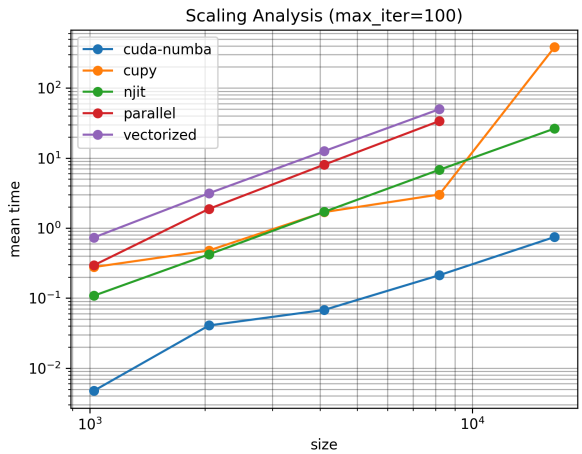


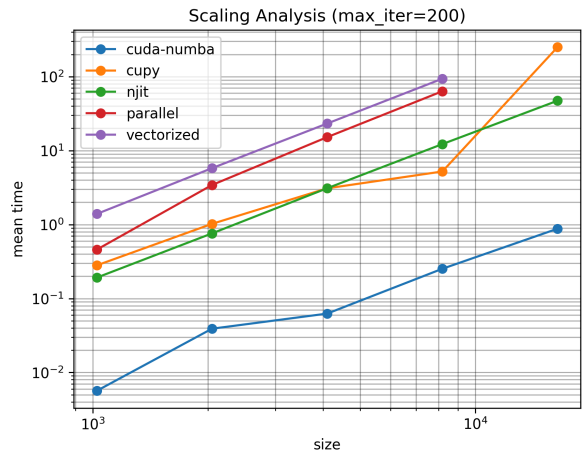


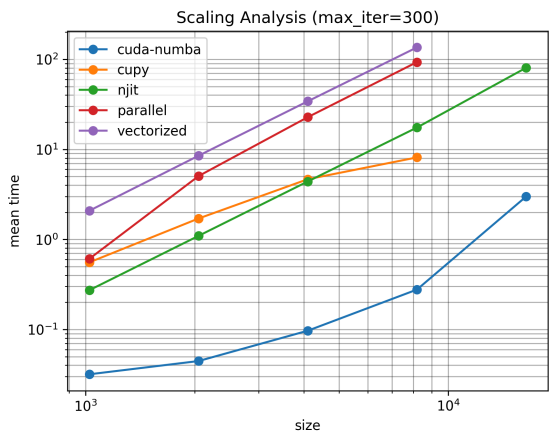


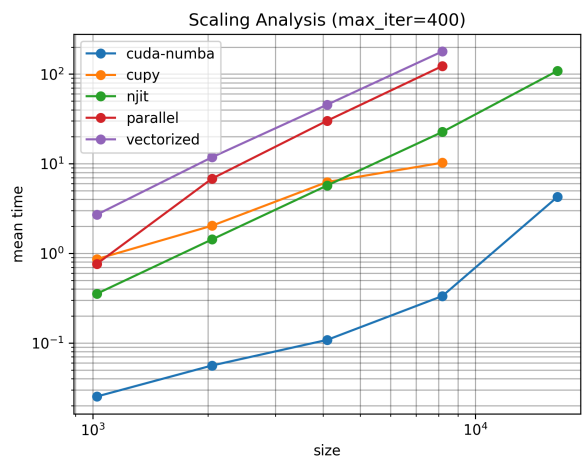


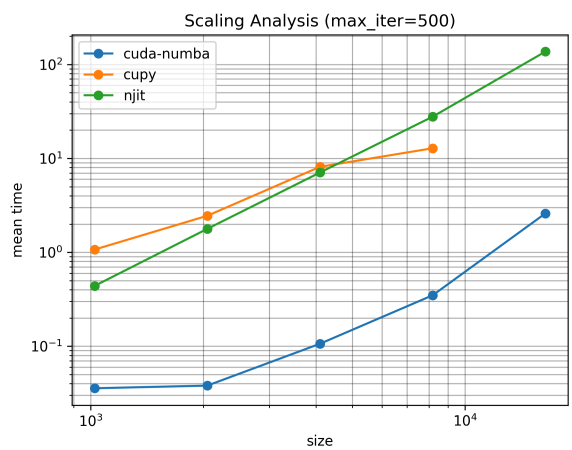
Scaling analysis (log-log) result across all tests 100-600 max iterations:

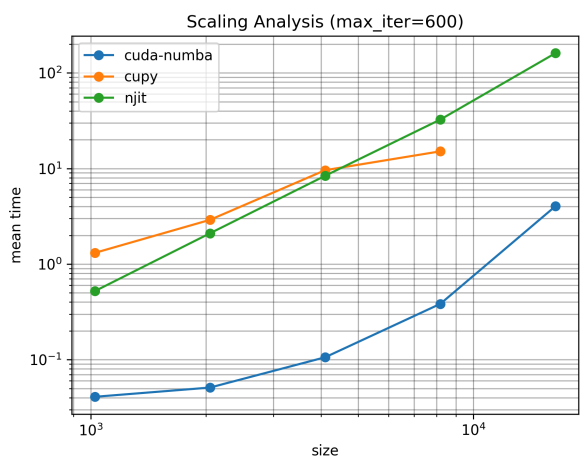












Evaluation of benchmarking results.

Cupy-numba outperformed everything so far seen in this course. The performance of the Cupy numba was also accelerated by me having it run on my home desktop which is equipped with an RTX 3090. With a powerful GPU with many cuda cores and VRAM the processes runned in the GPU can be executed very fast and efficiently.

Cupy had an interesting development as it initially didn't outperform njit, then did and eventually when the grid got too large, exploded in complexity.

This is likely caused by memory pressure: once the problem exceeds what fits efficiently in GPU VRAM or the L2 cache, memory bandwidth becomes the bottleneck, and the overhead of managing large allocations on the device causes execution time to spike.

After it took forever to terminate on grid size 16384x16384 with max iterations of 100 and 200 i decided to terminate the rest of the tests planned for cupy.